

Model Interpretability - SHAP Values

Jingrui Mu (261019374)

April 2026

Abstract

As machine learning models are increasingly used in high-stakes applications, model explainability has become essential for understanding, diagnosing, and communicating predictive decisions. This report reviews the motivation and methodology behind SHAP, a widely used framework for feature attribution. I begin by discussing why black-box models require interpretable explanations and introduce LIME as an early local surrogate explanation method. I then present the game-theoretic foundation of SHAP through Shapley values and explain how the SHAP framework connects additive feature attribution with desirable fairness and consistency properties. Several extensions, including KernelSHAP, and DeepSHAP, are discussed to illustrate how SHAP has been adapted to different model classes.

In addition to reviewing the methodology, this report examines how SHAP values should be interpreted in practice. Through simulation studies under both linear and nonlinear data-generating mechanisms, I illustrate that SHAP explanations are model-dependent: they describe how a fitted model uses the predictors rather than directly identifying the true data-generating process. I further discuss statistical inference for SHAP, including recent approaches for confidence intervals for global SHAP importance and I propose a potential bootstrap-based strategy for quantifying uncertainty in estimated SHAP curves.

1 Introduction

Background and motivation. As machine learning and artificial intelligence models are increasingly used in high-stakes domains such as medicine, finance, and policy-making, model interpretability has become an important topic. Modern predictive models, especially ensemble methods and deep neural networks, can achieve strong predictive performance but often behave as black boxes. In many applications, prediction accuracy alone is not sufficient: practitioners also want to understand why a model makes a certain prediction, which variables drive the prediction, and whether the model relies on scientifically meaningful or potentially misleading patterns. Therefore, model explainability is needed not only for improving transparency and trust, but also for model diagnosis, scientific discovery, feature selection, and responsible decision-making.

From LIME to SHAP. One influential local explanation method is LIME, which explains a black-box model by fitting a simple interpretable surrogate model around a target observation. The main idea is to perturb the input near a given point and fit a locally weighted linear model to approximate the black-box prediction function. Although LIME is intuitive and flexible, its explanations can be sensitive to the choice of perturbation distribution, kernel bandwidth, and local sampling scheme. As a result, the explanation may not be stable, and different choices in the local approximation procedure can lead to different feature attributions. These limitations motivated the development of more principled additive explanation methods.

SHAP and Shapley values. SHAP provides a unified framework for feature attribution by connecting local model explanations with Shapley values from cooperative game theory. In the Shapley-value formulation, each feature is viewed as a player in a cooperative game, and the prediction is viewed as the value generated by a coalition of features. The Shapley value assigns each feature its average marginal contribution over all possible feature subsets, giving a principled way to distribute the prediction among the input variables. Although Shapley values had been considered before in the context of variable importance, they were not widely used in machine learning practice because exact computation is expensive and the connection to practical explanation methods was not yet fully developed. The SHAP framework made Shapley-value-based explanations much more popular by providing a clear additive explanation formulation, connecting Shapley values with desirable explanation properties, and proposing computational tools for practical implementation.

Extensions of SHAP. After the SHAP framework was introduced, several specialized algorithms were developed to make Shapley-based explanations efficient for different model classes. For tree-based models, TreeSHAP exploits the structure of decision trees to compute exact or efficient SHAP values for random forests and gradient-boosted trees. For neural networks, DeepSHAP combines ideas from DeepLIFT and SHAP to approximate Shapley-value explanations in deep learning models. These developments significantly expanded the applicability of SHAP, making it one of the most widely used tools for explaining complex predictive models. As a result, SHAP is now commonly used to produce both local explanations, which describe the contribution of each feature to an individual prediction, and global importance summaries, such as mean absolute SHAP values, which rank features according to their overall contribution across a dataset.

Structure of the report. The remainder of this report is organized as follows. Section 2 introduce the main methodological background for model explanation, including LIME, Shapley values, the SHAP framework, and several important SHAP variants such as KernelSHAP,

and DeepSHAP. Section 3 presents simulation studies under both linear and nonlinear data-generating mechanisms to compare how SHAP explanations change across different fitted models. Section 4 discusses statistical inference for global SHAP importance, including recent debiased approaches for constructing confidence intervals, and propose a bootstrap-based strategy for estimating the uncertainty of the SHAP curve.

2 Material Methods

2.1 Shapley Values

The idea of SHAP comes from cooperative game theory. In that setting, we consider a set of players $N = \{1, \dots, p\}$. For every subset $S \subseteq N$, a value function $v(S)$ is defined to represent the total payoff that the players in S can achieve by cooperating. The quantity $v(\emptyset)$ is taken as the baseline payoff, namely the value obtained when no player participates. Shapley (1953) [4] introduced the Shapley value as a principled way to distribute the total payoff among players according to their contributions. A key point is that a player’s contribution cannot, in general, be determined by fixing one subset of the other players and simply comparing the payoff with and without that player. The reason is that the marginal contribution of a player may depend on which other players are already present. Therefore, cooperative game theory does not focus on a single comparison, but instead considers the player’s marginal contribution across all possible subsets of the remaining players. To make this idea concrete, consider the case of three players $N = \{A, B, C\}$ and then we possibly can have the following profits in different subsets: $v(\emptyset) = 0$, $v(\{A\}) = 1$, $v(\{B\}) = 2$, $v(\{C\}) = 0$, $v(\{A, B\}) = 10$, $v(\{A, C\}) = 1$, $v(\{B, C\}) = 2$, $v(\{A, B, C\}) = 10$. For player A , the marginal contribution depends on the subset to which A is added. For example, $v(\{A\}) - v(\emptyset) = 1$, while $v(\{A, B\}) - v(\{B\}) = 8$. Thus, the contribution of player A is not a single fixed quantity; it varies with the subset under consideration. The Shapley value resolves this by averaging the marginal contribution of a player over all possible subsets, thereby providing a fair and mathematically justified allocation of the total payoff.

This idea can be naturally transferred to machine learning. For a fixed observation, the model prediction can be viewed as the outcome of a cooperative game. In this analogy, the players are the feature values of that observation, and the payoff is the model prediction relative to a baseline, often taken as the prediction minus the average prediction over all observations. Each feature contributes to the final prediction by interacting with the others, just as each player contributes to the total profit in a coalition.

Under this interpretation, the Shapley value measures how much each feature contributes to the prediction. More precisely, it is defined as the average marginal contribution of that feature over all possible subsets of features. In other words, for each possible coalition, we look at how much the prediction changes when a given feature is added, and then average this contribution across all possible coalitions. This provides a fair and mathematically grounded way to attribute the prediction to individual features.

For example, we predict the price of a certain apartment is 400,000\$, and the average prediction for all apartments in the dataset is 380,000\$. The goal is to explain how each of these feature values contributed to the prediction compared to the average prediction. The feature values are park-nearby, top high school nearby, area of 50 m^2 , floor-2nd. They work together to achieve the prediction of 400,000 \$. The answer could be: top-school-nearby contributed 30,000\$; park-nearby contributed 10,000\$, floor-2nd contributed -10,000\$ and area of 50 m^2 contributed -10,000\$. The contributions add up to 20,000\$, which is the final prediction of the certain apartment minus the average predicted apartment price. The Shapley value is the average marginal contribution of a feature value across all possible coalitions. If we intend to

analyze the marginal contribution of the top-school-nearby feature value, we need to go through all possible subsets excluded top-school-nearby feature. These possible subsets could be: $\{\}$ (empty subset), $\{\text{park-nearby}\}$, $\{\text{area of } 50 \text{ m}^2\}$, $\{\text{floor-2nd}\}$, $\{\text{park-nearby, area of } 50 \text{ m}^2\}$, $\{\text{park-nearby, floor-2nd}\}$, $\{\text{area of } 50 \text{ m}^2, \text{floor-2nd}\}$, $\{\text{park-nearby, area of } 50 \text{ m}^2, \text{floor-2nd}\}$. For each of these subsets, we compute the predicted price with and without the feature value **top-school-nearby** through that prediction model and take the difference to get the marginal contribution. The Shapley value is the (weighted) average of all marginal contributions. The feature values of features that are not in a subset with random feature values from a background dataset to get a prediction from the prediction model, where the background dataset is sampled from the whole dataset.

The Shapley value of a feature value is its contribution to the payout, weighted and summed over all possible feature value combinations:

$$\phi_j(val_{\mathbf{x}_i}) = \sum_{S \subseteq \{1, \dots, p\} / \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} (val_{\mathbf{x}_i}(S \cup \{j\}) - val_{\mathbf{x}_i}(S)) \quad (1)$$

where S is a subset of the features used in the model, $\mathbf{x}$ is the vector of feature values of the instance to be explained, and p is the total number of features. $val_{\mathbf{x}}(S)$ is the prediction for feature values in set S that marginalized over features X_C , which are all the features not included in S ($S \cap C = \emptyset$ and $S \cup C = \{1, \dots, p\}$):

$$val_{\mathbf{x}_i}(S) = \int \hat{f}(x_S^{(i)}, x_C) dP_{X_C}(x_C) - E[\hat{f}(X)] \quad (2)$$

where x_S is the observed feature values of the instance being explained, restricted to S ; X_C is the remaining features, treated as random under the background distribution. So in practice, the first term in (2) is obtained by averaging over background samples $1, \dots, M$:

$$\int \hat{f}(x_S^{(i)}, x_C) dP_{X_C}(x_C) \simeq \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_C^{(m)})$$

Therefore,

$$\begin{aligned} val_{\mathbf{x}_i}(S) &\simeq \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_C^{(m)}) - \frac{1}{M} \sum_{m=1}^M \hat{f}(\mathbf{x}^{(m)}) \\ val_{\mathbf{x}_i}(S \cup \{j\}) &\simeq \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_j^{(i)}, x_C^{(m)}) - \frac{1}{M} \sum_{m=1}^M \hat{f}(\mathbf{x}^{(m)}) \end{aligned}$$

Thus, $val_{\mathbf{x}_i}(S \cup \{j\}) - val_{\mathbf{x}_i}(S) = \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_j^{(i)}, x_C^{(m)}) - \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_C^{(m)})$

The Shapley value is the method that satisfies the properties **Efficiency**, **Symmetry**, **Dummy** and **Additivity**.

1. **Efficiency**: The feature contributions can add up to the difference of prediction for $\mathbf{x}_i$ and the average: $\sum_{j=1}^p \phi_j(\mathbf{x}_i) = \hat{f}(\mathbf{x}_i) - E[\hat{f}(X)]$. **Note**: In cooperative game theory, "efficiency" means the entire payoff is fully allocated among these players, with nothing lost and nothing left over. In prediction, it means that the feature attributions explain the whole deviation of the prediction from baseline, where baseline is $E[\hat{f}(X)]$.
2. **Symmetry**: The contributions of two feature values j and k should be the same if they contribute equally to all possible subsets. if $val_{\mathbf{x}_i}(S \cup \{j\}) = val_{\mathbf{x}_i}(S \cup \{k\})$ for

all $S \subseteq \{1, \dots, p\} \setminus \{j, k\}$, then $\phi_j(\mathbf{x}_i) = \phi_k(\mathbf{x}_i)$. The Shapley value averages a feature's marginal contribution over all possible subsets. If feature j and k always make exactly the same contribution in every subset, then every term in the average for j matches the corresponding term in the average for k .

3. **Dummy:** A feature j that does not change the predicted value - regardless of which subset of feature values it is added to - should have a Shapley value of 0. If $val_{\mathbf{x}_i}(S \cup \{j\}) = val_{\mathbf{x}_i}(S)$ for all $S \subseteq \{1, \dots, p\}$ then $\phi_j(\mathbf{x}_i) = 0$. This follows directly from the Shapley formula, since every marginal contribution term $val_{\mathbf{x}_i}(S \cup \{j\}) - val_{\mathbf{x}_i}(S)$ is equal to zero, and therefore their weighted average is also zero.
4. **Additivity:** Let $val_{\mathbf{x}_i}$ and $val_{\mathbf{x}_i}^+$ be two set value functions defined on the same feature set $\{1, \dots, p\}$. Consider the combined game $val_{\mathbf{x}_i} + val_{\mathbf{x}_i}^+(S) = val_{\mathbf{x}_i}(S) + val_{\mathbf{x}_i}^+(S)$, $\forall S \subseteq \{1, \dots, p\}$. Then the corresponding Shapley value for feature j satisfies $\phi_j^{val+val^+}(\mathbf{x}_i) = \phi_j(\mathbf{x}_i) + \phi_j^+(\mathbf{x}_i)$

How to obtain Shapley Values? All possible subsets of feature values have to be evaluated with and without the j -th feature to calculate the exact Shapley value. However, there is a calculation problem as we have seen in the lectures: calculating the exact value as in (1) is pretty challenging as the number of possible subsets exponentially increases as more features are added. Strumbelj and Kononenko (2014) [5] proposed an approximation with Monte-Carlo sampling:

$$\hat{\phi}_j(val_{\mathbf{x}_i}) = \frac{1}{B} \sum_{b=1}^B (\hat{f}(\mathbf{x}_{+j}^{(b)}) - \hat{f}(\mathbf{x}_{-j}^{(b)})) \quad (3)$$

For each repetition b , two random steps are performed: (1) a random permutation of the features is generated, and (2) a random background observation is drawn from the data. The random permutation determines which features appear before feature j . These features form the subset S . Based on this subset, two hybrid input vectors are constructed. The vector $\mathbf{x}_{+j}^{(b)}$ is formed as follows: the features in S take their observed values from the instance $\mathbf{x}_i$, the feature of interest j also takes its observed value from $\mathbf{x}_i$, and all remaining features are filled in using the values from the sampled background observation. Similarly, the vector $\mathbf{x}_{-j}^{(b)}$ is constructed by letting the features in S' take their observed values from the instance $\mathbf{x}_i$, while feature j and all remaining features are replaced by the corresponding values from the sampled background observation. The algorithm is summarized as follows:

Algorithm 1 Monte Carlo approximation of the Shapley value for feature j at instance $\mathbf{x}_i$

- 1: **Input:** Number of resamples B , instance of interest $\mathbf{x}_i$, feature index j , data matrix X , and fitted prediction function $\hat{f}$
 - 2: **Output:** Approximate Shapley value $\phi_j(\mathbf{x}_i)$
 - 3: **for** $b = 1, \dots, B$ **do**
 - 4: Draw a random background observation $\mathbf{x}^{(b)}$ from X
 - 5: Draw a random permutation $\mathbf{o} = ((1), \dots, (p))$ of the feature indices and Let $S^{(b)} = \{(1), \dots, (j-1)\}$ denote the set of features that appear before feature j in the permutation
 - 6: Construct these two input vectors $\mathbf{x}_{+j}^{(b)} = (x_{(1)}, \dots, x_{(j-1)}, x_{(j)}, x_{(j+1)}^{(b)}, \dots, x_p^{(b)})$ and $\mathbf{x}_{-j}^{(b)} = (x_{(1)}, \dots, x_{(j-1)}, x_{(j)}^{(b)}, x_{(j+1)}^{(b)}, \dots, x_p^{(b)})$
 - 7: Compute the sampled marginal contribution $\phi_j^{(b)}(\mathbf{x}_i) = \hat{f}(\mathbf{x}_{+j}^{(b)}) - \hat{f}(\mathbf{x}_{-j}^{(b)})$
 - 8: **end for**
 - 9: Compute the Monte Carlo estimate of the Shapley value $\phi_j(\mathbf{x}_i) = \frac{1}{B} \sum_{b=1}^B \phi_j^{(b)}(\mathbf{x}_i)$
-

Limitations The calculation of Shapley value requires a lot of computing time. An exact computation of the Shapley value is computationally expensive because there are 2^k possible subsets of the feature values and the "absense" of a feature has to be simulated by drawing random instances, which increases the variance for the estimate of the Shapley values estimation. The exponential number of the coalitions is dealt with by sampling coalitions and limiting the number of resamples B . Decreasing B reduces the computation time, but increases the variance of the Shapley value.

2.2 LIME

Local surrogate models are interpretable models that are used to explain individual predictions of black box machine learning models. Ribeiro, Singh and Guestrin (2016) [3] proposed local interpretable model-agnostic explanations (LIME), which is an approach for fitting surrogate models. The intuition of the idea is to train an interpretable model to approximate of the machine learning model predictions locally. Local surrogate models with interpretability constraint can be expressed as

$$\text{explanation}(\mathbf{x}_i) = \operatorname{argmin}_{g \in G} L(\hat{f}, g, \pi_{\mathbf{x}_i}) + \Omega(g) \quad (4)$$

The explanation model for instance $\mathbf{x}_i$ is the model g that minimizes loss L , which measures how close the explanation is to the prediction of the original machine learning model $\hat{f}$, while the model complexity $\Omega(g)$ is kept low. G is the family of possible explanations. The proximity measure $\pi_{\mathbf{x}_i}$ defines how large the neighborhood around instance $\mathbf{x}_i$ is that we consider for the explanation. Ribeiro, Singh and Guestrin (2016) [3] used the following Figure 1 to represent the high-level idea of LIME.

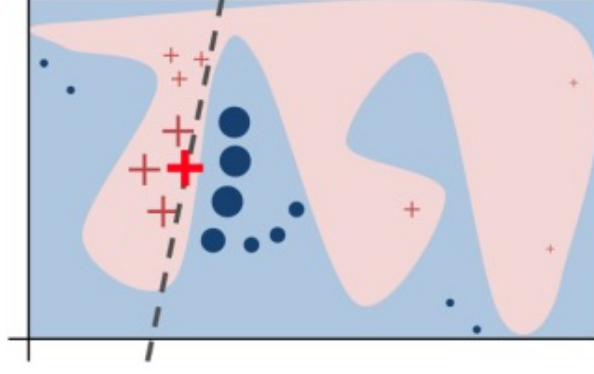


Figure 1: Toy Example to present intuition of LIME

The blue and pink background represents the complex underlying prediction function, which is assumed to be unknown and too complicated to be approximated well by a simple interpretable model on a global scale. The bold red cross denotes the instance of interest. The remaining red crosses correspond to sampled instances in its neighborhood, with their weights determined by their proximity to the instance being explained; larger points indicate higher weights. The dashed line shows the fitted local surrogate model, which provides an interpretable approximation of the black-box model in the vicinity of the selected instance.

Sampling for Local Exploration As shown in Figure 1, the main goal of LIME is to minimize the locality-aware loss $L(\hat{f}, g, \pi_{\mathbf{x}_i})$. Thus, in order to learn the local behavior of $\hat{f}$ as the interpretable inputs vary, they approximate $L(\hat{f}, g, \pi_{\mathbf{x}_i})$ by drawing sample, weighted by $\pi_{\mathbf{x}_i}$. For different types of dataset, there are different ways of sampling given:

1. **Tabular data:** Tabular data is data that comes in tables, with each row representing a data point and each column a feature. Suppose the data point to explain is $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$. For continuous feature j , sample a perturbed value from a simple distribution based on the training data: $\tilde{x}_j = \mu_j + \sigma_j Z_j$, where $Z_j \sim N(0, 1)$ and μ_j, σ_j are the empirical mean and standard deviation. For categorical feature j , sample a category according to its empirical frequency in the training data: $P(Z_j = c) = \hat{p}_{jc}$.
2. **Text data:** LIME does not work directly on raw text. It converts data into an interpretable binary vector and they create a neighborhood around the original text instance by randomly "turning off" words. Suppose the original text has m interpretable words: $\mathbf{x}_i = (1, \dots, 1) \in (0, 1)^m$, where all words in the original text $\mathbf{x}_i$ are present. A perturbed sample is a binary vector $\mathbf{x}'_i = (x'_{i1}, \dots, x'_{im})$, $x'_{ij} \in \{0, 1\}$, where $x'_{ij} = 1$ means keeping word j and $x'_{ij} = 0$ means removing word j . The way on how to get sampled binary vector $\mathbf{x}'_i$ consists of two steps. At first, sample an integer from $K \in \{1, \dots, m\}$ uniformly to choose how many words will be present. Secondly, Given $K = k$, they sample uniformly among the m words to get a size of k words present.
3. **Image data:** LIME does not treat every pixel in the image as a feature, instead groups nearby similar pixels into regions. Each region or superpixel is then treated as one interpretable feature. Suppose an image can be segmented as d superpixels or regions, the interpretable representation is $\mathbf{x}_i = (1, \dots, 1) \in \{0, 1\}^d$. And then draw the perturbed sample following the steps in text data.

Sparse Linear Explanations using LIME After sampling data points around the instance $\mathbf{x}_i$ to explain, they proposed sparse linear explanations to interpret the prediction for $\mathbf{x}_i$. The algorithm is summarized as follows:

Algorithm 2 Sparse Linear Explanations using LIME

- 1: **Input:** Number of samples N , instance of interest $\mathbf{x}_i$, similarity kernel $\pi_{\mathbf{x}_i}$ and fitted prediction function $\hat{f}$
 - 2: **Output:** Sparse Linear Explanation Model $\hat{g}$
 - 3: **for** $i = 1, \dots, N$ **do**
 - 4: $\mathbf{x}'_i \leftarrow \text{sample_around}(\mathbf{x}_i)$ and calculate weights $\pi_{\mathbf{x}}$ for each $\mathbf{x}'_i$
 - 5: **end for**
 - 6: $\hat{g} \leftarrow \operatorname{argmin}_{g \in G} L(\hat{f}, g, \pi_{\mathbf{x}_i}) + \Omega(g)$, where G is the class of linear models such that $g(\mathbf{x}'_i) = \mathbf{w}_g \cdot \mathbf{x}'_i$, $\pi_{\mathbf{x}_i}(\mathbf{x}'_i) = \exp(-D(\mathbf{x}_i, \mathbf{x}'_i)^2 / \sigma^2)$ and $\Omega(g) = I\{\|\mathbf{w}_g\|_0 > K\}$
-

K is selected by the user and usually adapted to be as big as the user can handle.

Limitations The choice of neighborhood is an unsolved problem when using LIME, especially with tabular data. As discussed in **Sampling for Local Exploration**, data points are sampled from a Gaussian distribution, ignoring the correlation between features. This can lead to unlikely data point, which can then be used to learn local explanation models. Another issue is the instability of the explanations. Alvarez-Melis and Jaakkola (2018) [1] showed that the explanations of two very close points varied greatly in a simulated setting. Also, if you repeat the sampling process, then the explanations that come out can be different.

2.3 SHAP

Lloyd Shapley first introduced the concept of Shapley values in the context of cooperative game theory. Strumbelj and Kononenko (2014) [5] extended this idea to the explanation of machine learning predictions. However, this idea did not gain widespread attention at that time. Later, Lundberg and Lee (2017) [2] proposed SHAP, which is a new way to estimate Shapley values for interpreting complex machine learning predictions, along with a theory connecting Shapley values with LIME. Recall the efficiency property of Shapley values in Section 2.1, Shapley value explanation can be represented as an additive feature attribution method, a linear model, $\sum_{j=1}^p \phi_j + E[\hat{f}(X)] = \hat{f}(\mathbf{x}_i)$. This view connects LIME and Shapley values. SHAP specifies the explanation as

$$g(\mathbf{x}'_i) = \phi_0 + \sum_{j=1}^p \phi_j x'_{ij} \quad (5)$$

where g is the explanation model, $\mathbf{x}'_i = (x'_{i1}, \dots, x'_{ip})^T \in \{0, 1\}^p$, p is the total number of features, and ϕ_j is the feature attribution for feature j . $\mathbf{x}'_i$ is the simplified features. For $\mathbf{x}_i$, the instance of interest, the corresponding simplified features $\mathbf{x}'_i$ is a vector of all 1's and 0's. If $x_{ij} = 1$, it means the feature j is present otherwise not. SHAP not only satisfies these properties discussed in Section 2.1, Efficiency, Symmetry, Dummy and Additivity, but also enjoys other three properties shown in Lundberg and Lee (2017) [2].

1. Local Accuracy: $\hat{f}(\mathbf{x}_i) = g(\mathbf{x}'_i) = \phi_0 + \sum_{j=1}^p \phi_j x'_{ij} = E[\hat{f}(X)] + \sum_{j=1}^p \phi_j$. The explanation model $g(\mathbf{x}')$ matches the original model $\hat{f}(\mathbf{x}_i)$.
2. Missingness. If feature j is absent in the simplified input representation, then its attribution must be zero.

3. Consistency: Let $\hat{f}_{\mathbf{x}}(\mathbf{z}') = \hat{f}(h_{\mathbf{x}}(\mathbf{z}'))$ and $\mathbf{z}'_{-j}$ indicates that $z'_j = 0$. For any two prediction models $\hat{f}$ and $\hat{f}'$ that satisfy

$$\hat{f}'_{\mathbf{x}}(\mathbf{z}') - \hat{f}'_{\mathbf{x}}(\mathbf{z}'_{-j}) \geq \hat{f}_{\mathbf{x}}(\mathbf{z}') - \hat{f}_{\mathbf{x}}(\mathbf{z}'_{-j})$$

for all inputs $\mathbf{z}' \in \{0, 1\}^p$, then: $\phi_j(\hat{f}', \mathbf{x}) \geq \phi_j(\hat{f}, \mathbf{x})$. $\hat{f}_{\mathbf{x}}(\mathbf{z}') = \hat{f}(h_{\mathbf{x}}(\mathbf{z}'))$ means taking the simplified binary vector $\mathbf{z}$ and mapping it back to a valid original input using $h_{\mathbf{x}}$, and then evaluating the prediction model there. Consistency compares two models, $\hat{f}$ and $\hat{f}'$. If for every subset $\mathbf{z}$, the contribution of feature j becomes larger in the new model, then the attribution for feature j should not decrease.

Lundberg and Lee (2017) [2] stated that only one possible explanation model g following (5) and satisfies Properties local accuracy, missingness and consistency:

$$\phi_j(f, \mathbf{x}) = \sum_{\mathbf{z}'} \frac{|\mathbf{z}'|!(p - |\mathbf{z}'| - 1)!}{p!} [f_{\mathbf{x}}(\mathbf{z}') - f_{\mathbf{x}}(\mathbf{z}' / j)] \quad (6)$$

where $|\mathbf{z}'|$ is the number of non-zero entries in $\mathbf{z}'$, and $\mathbf{z}' \in \{0, 1\}^p$.

2.3.1 KernelSHAP

Lundberg and Lee (2017) [2] proposed several SHAP estimations. This section focuses on KernelSHAP, which is the combination of Linear LIME and Shapley values. Linear LIME uses a linear model to locally approximate $\hat{f}$ given a specific instance $\mathbf{x}$. Recall (4) in Section 2.2, LIME is to minimize the locality-aware loss $L(\hat{f}, g, \pi_{\mathbf{x}_i})$. Under (5), they specified forms of $\pi_{\mathbf{x}_i}$, L and Ω that make solutions of equation (4) consistent with properties local accuracy, missingness, and consistency:

$$\Omega(g) = 0 \quad (7)$$

$$\pi_{\mathbf{x}_i}(\mathbf{z}'_i) = \frac{(p-1)}{\binom{p}{|\mathbf{z}'_i|} |\mathbf{z}'_i| (p - |\mathbf{z}'_i|)} \quad (8)$$

$$L(\hat{f}, g, \pi_{\mathbf{x}_i}) = \sum_{\mathbf{z}'_i \in Z} [\hat{f}(h_{\mathbf{x}_i}^{-1}(\mathbf{z}'_i)) - g(\mathbf{z}'_i)]^2 \pi_{\mathbf{x}_i}(\mathbf{z}'_i) \quad (9)$$

where $|\mathbf{z}'_i|$ represents the number of non-zero elements in $\mathbf{z}'_i$. Note that $\pi_{\mathbf{x}_i}(\mathbf{z}') = \infty$ when $|\mathbf{z}'_i| \in \{0, p\}$, which enforces $\phi_0 = \hat{f}_{\mathbf{x}}(\emptyset)$ and $\hat{f}(\mathbf{x}_i) = \sum_{j=1}^p \phi_j$

The KernelSHAP estimation can be summarized as the following algorithm:

Algorithm 3 Kernel SHAP for one observation x

- 1: **Input:** fitted model f , instance $\mathbf{x}$, background dataset $\{x^{(1)}, \dots, x^{(M)}\}$, number of sampled coalitions K
 - 2: **Output:** estimated SHAP values $\hat{\phi}_1, \dots, \hat{\phi}_p$
 - 3: **for** $k = 1, \dots, K$ **do**
 - 4: Sample a coalition vector $x'^{(k)} \in \{0, 1\}^p$
 - 5: **for** $m = 1, \dots, M$ **do**
 - 6: Construct a hybrid input $\tilde{x}^{(k,m)}$ by setting and evaluate $f(\tilde{x}^{(k,m)})$
$$\tilde{x}_j^{(k,m)} = \begin{cases} x_j, & z_j'^{(k)} = 1, \\ x_j^{(m)}, & z_j'^{(k)} = 0, \end{cases} \quad j = 1, \dots, p$$
 - 7: **end for**
 - 8: Compute $y^{(k)} = \frac{1}{M} \sum_{m=1}^M f(\tilde{x}^{(k,m)})$ and let $s_k = |x'^{(k)}|$ and define $\pi_{\mathbf{x}}^{(k)} = \frac{p-1}{\binom{p}{s_k} s_k (p-s_k)}$
 - 9: **end for**
 - 10: Fit the weighted least-squares problem $\min_{\phi_0, \phi_1, \dots, \phi_p} \sum_{k=1}^K \pi_{\mathbf{x}}^{(k)} \left(y^{(k)} - \phi_0 - \sum_{j=1}^p \phi_j x_j'^{(k)} \right)^2$
 - 11: Return $\hat{\phi}_1, \dots, \hat{\phi}_p$
-

2.3.2 Deep SHAP (DeepLIFT + SHAP)

While Kernel SHAP is model-agnostic and can, in principle, be applied to any predictive model, including deep neural networks, it does not exploit the internal compositional structure of such models. This naturally raises the question of whether the layered architecture of deep networks can be leveraged to obtain more computationally efficient SHAP explanations. To address this, Lundberg and Lee (2017) [2] proposed Deep SHAP, a SHAP-based explanation method tailored to deep neural networks. The key idea is to combine DeepLIFT-style backpropagation of feature contributions with the Shapley-value interpretation underlying SHAP, thereby enabling efficient attribution propagation through the network while preserving the additive explanation framework.

DeepLIFT explains a neural network prediction by comparing the activation of each neuron at the input $\mathbf{x}_i$ to its activation at a reference input $\mathbf{x}'_i$. For a corresponding output y_i , DeepLIFT considers the difference $\Delta y_i = y(\mathbf{x}_i) - y(\mathbf{x}'_i)$. Then it propagates this difference backward through the network and assigns contribution scores $C_{\Delta x_j \rightarrow \Delta y}$ such that $\Delta f_i = f(\mathbf{x}_i) - f(\mathbf{x}'_i) = \sum_{j=1}^p C_{\Delta x_j \rightarrow \Delta f_i}$. So DeepLIFT is also an additive attribution method, but relative to a single reference point $\mathbf{x}'_i$, not necessarily the expected model output. In order to deliver the idea behind DeepLIFT better, I used the following two-layers neural network as reference:

Consider the neural network as above, $x_j \rightarrow a_h \rightarrow z_h \rightarrow f(x)$. Now choose a reference input $\mathbf{x}' = (x'_1, \dots, x'_p)^T$, and pass $\mathbf{x}'$ through the network gives

$$\begin{aligned} a'_h &= \sum_{j=1}^p w_{hj}^{(1)} x'_j + b_h^{(1)} \\ z'_h &= \sigma(z'_h) \\ f(\mathbf{x}') &= \sum_{h=1}^H w_h^{(2)} z'_h + b^{(2)} \end{aligned}$$

DeepLIFT explains the difference $\Delta f = f(\mathbf{x}) - f(\mathbf{x}')$ by assigning contributions to the original input features x_j . For the first linear layer, $\Delta a_h = a_h - a'_h = \sum_{j=1}^p w_{hj}^{(1)} (x_j - x'_j) = \sum_{j=1}^p w_{hj}^{(1)} \Delta x_j$.

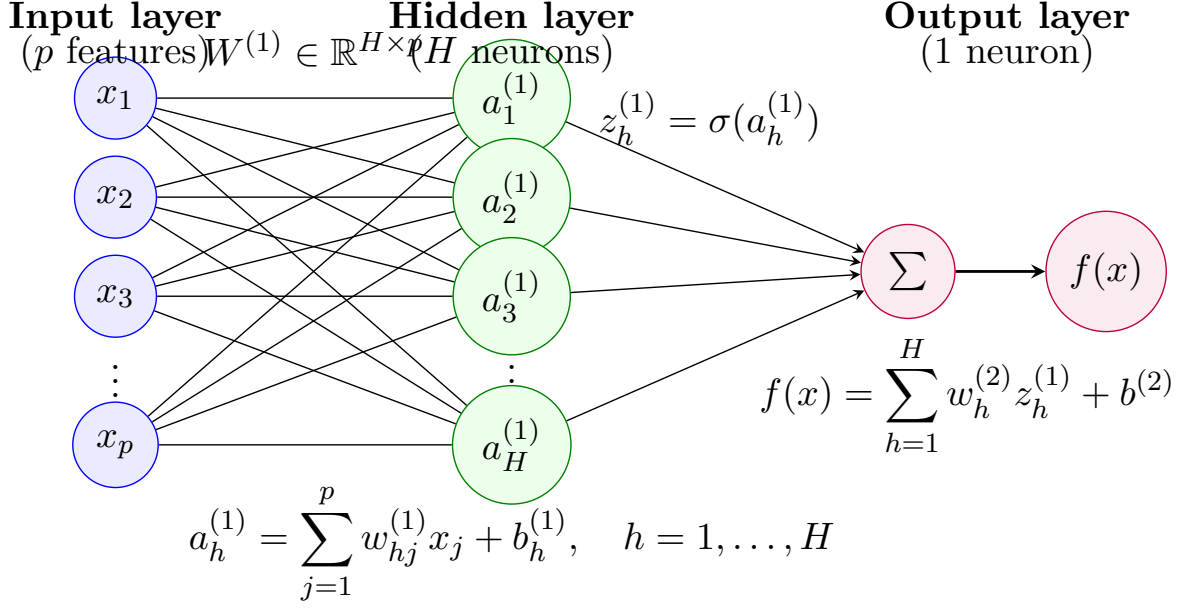


Figure 2: A two-layer fully connected neural network with one hidden layer.

So the contribution from input feature x_j to the pre-activation a_h is $C_{x_j \rightarrow a_h} = w_{hj}^{(1)} \Delta x_j$. The activation contribution from a_h to z_h is based on $\Delta z_h = \sigma(a_h) - \sigma(a'_h)$. DeepLIFT defines a multiplier $m_{a_h \rightarrow z_h} = \frac{\Delta z_h}{\Delta a_h} = \frac{\sigma(a_h) - \sigma(a'_h)}{a_h - a'_h}$ when $\Delta a_h \neq 0$. Therefore, the contribution from x_j to z_h is $C_{x_j \rightarrow z_h} = C_{x_j \rightarrow a_h} m_{a_h \rightarrow z_h} = w_{hj}^{(1)} (x_j - x'_j) \frac{\Delta z_h}{\Delta a_h}$. The contribution from hidden neuron z_h to the output is $C_{z_h \rightarrow f} = w_h^{(2)} \Delta z_h$. The multiplier is $m_{z_h \rightarrow f} = \frac{C_{z_h \rightarrow f}}{\Delta z_h} = w_h^{(2)}$. Thus the contribution from input feature x_j to the final output through hidden unit h is $C_{x_j \rightarrow f}^{(h)} = C_{x_j \rightarrow z_h} m_{z_h \rightarrow f} = w_h^{(2)} w_{hj}^{(1)} \Delta x_j \frac{\Delta z_h}{\Delta a_h}$. Finally, sum over all hidden neurons $h = 1, \dots, H$: $C_j = C_{x_j \rightarrow f} = \sum_{h=1}^H w_h^{(2)} w_{hj}^{(1)} \Delta x_j \frac{\Delta z_h}{\Delta a_h} = \sum_{h=1}^H w_h^{(2)} w_{hj}^{(1)} (x_j - x'_j) \frac{\sigma(a_h) - \sigma(a'_h)}{a_h - a'_h}$. C_j is the DeepLIFT attribution for feature j in this two-layer fully connected network.

Recall SHAP in previous discussion, for a prediction $f(\mathbf{x}_i)$, SHAP wants $f(\mathbf{x}_i) - E[f(X)] = \sum_{j=1}^p \phi_j(\mathbf{x}_i)$, where $\phi_j(\mathbf{x}_i) = \sum_{S \subseteq [p]/j} \frac{|S|!(p-|S|-1)!}{p!} [v(S \cup \{j\}) - v(S)]$ and $v(S) = E_{X'_{Sc}} [f(x_S, X'_{Sc})]$. In the original SHAP idea, missing features are replaced using background samples. However, this computation is hugely expensive in deep neural networks. Therefore, DeepSHAP borrows the idea of DeepLIFT to obtain $\phi_j(\mathbf{x}_i)$:

1. Choose background samples $\{x'^{(1)}, \dots, x'^{(B)}\}$. These background samples represent the baseline distribution. The background dataset is used to approximate the expectation term, and the final attributions sum to $f(\mathbf{x}_i) - \frac{1}{B} \sum_{b=1}^B f(\mathbf{x}'^{(b)}) \simeq \sum_{j=1}^p \phi_j(\mathbf{x}_i)$
2. For each background point, run the target input and the background input through the network following the DeepLIFT idea above.
3. For each background data point $x'^{(b)}$, we can get $C_{x_j \rightarrow f}^{(b)} = \sum_{h=1}^H w_h^{(2)} w_{hj}^{(1)} \Delta x_j^{(b)} \frac{\Delta z_h^{(b)}}{\Delta a_h^{(b)}}$. For this simple two-layer network, the practical DeepSHAP attribution is approximately $\phi_j(\mathbf{x}_i) = \frac{1}{B} \sum_{b=1}^B C_{x_j \rightarrow f}^{(b)}$. That is, $\phi_j(\mathbf{x}_i) = \frac{1}{B} \sum_{b=1}^B \sum_{h=1}^H w_h^{(2)} w_{hj}^{(1)} (x_j - x_j'^{(b)}) \frac{\sigma(a_h) - \sigma(a_h'^{(b)})}{a_h - a_h'^{(b)}}$

Thus DeepSHAP avoids enumerating all possible subset options. DeepSHAP is not explicitly computing all $v(S)$, instead using DeepLIFT backpropagation rules that approximate the effect of Shapley averaging.

3 Simulation Results

This section is focusing on how to present the SHAP results and compare the results under linear and nonlienar scenarios, which indicates that SHAP-based methods are model-agnostic approach to explain complex models.

3.1 Linear-based

Low-dimensional scenario:

- **Setting:** We simulate data from a linear Gaussian model with

$$n = 500, \quad p = 5, \quad n_{\text{reps}} = 100.$$

- **Covariates:** Generate

$$X_i \sim N_p(\mu, \Sigma), \quad i = 1, \dots, n,$$

where

$$\mu = \mathbf{0}_p, \quad \Sigma_{jk} = \rho^{|j-k|}, \quad 1 \leq j, k \leq p.$$

The parameter ρ controls the correlation among predictors.

- **Outcome model:** Given X_i , generate

$$Y_i = \beta_0 + X_i^\top \beta + \varepsilon_i,$$

with

$$\beta_0 = 1.5, \quad \beta = (0.5, 2.0, -1.0, 1.5, -1.0)^\top,$$

and

$$\varepsilon_i \sim N(0, 1).$$

- **Replication scheme:** Since we are analyzing each feature j's contribution of each data point $\mathbf{x}_i$, the design matrix $X \in \mathbb{R}^{500 \times 5}$ is generated once and fixed across all replications. For each replication, only the noise vector ε is regenerated, producing a new response vector Y .

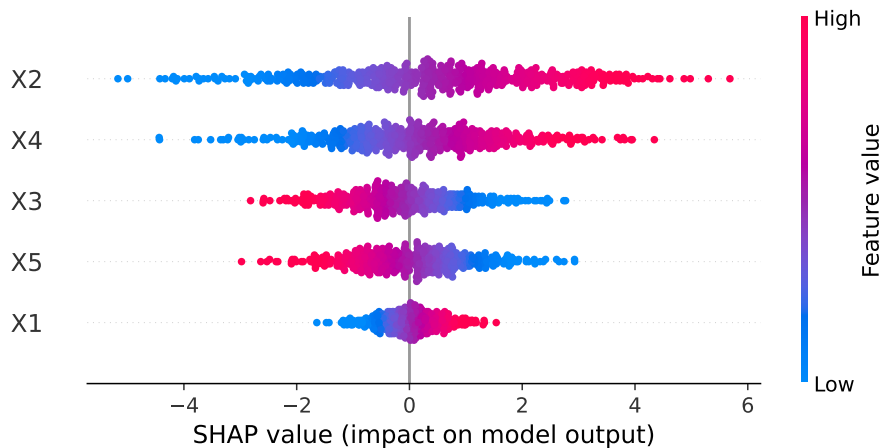


Figure 3: Beeswarm plot ($p=5$, $n=500$)

- The beeswarm plot summarizes the distribution of SHAP values for each predictor, where points to the right increase the model prediction and points to the left decrease it.
- The most influential variables are X_2 , X_4 , X_3 , and X_5 , while X_1 has the smallest overall impact. This agrees with the true coefficient magnitudes:

$$|\beta_2| = 2.0, \quad |\beta_4| = 1.5, \quad |\beta_3| = |\beta_5| = 1.0, \quad |\beta_1| = 0.5.$$

- For X_2 and X_4 , high feature values are mostly associated with positive SHAP values, indicating positive effects on the predicted response. This matches the positive coefficients:

$$\beta_2 = 2.0, \quad \beta_4 = 1.5.$$

- For X_3 and X_5 , high feature values are mostly associated with negative SHAP values, while low feature values are associated with positive SHAP values. This reflects their negative coefficients:

$$\beta_3 = -1.0, \quad \beta_5 = -1.0.$$

- X_1 has a narrower SHAP-value spread, suggesting a weaker contribution to prediction, consistent with its smaller coefficient:

$$\beta_1 = 0.5.$$

- The approximately symmetric horizontal spread around zero reflects the Gaussian covariate distribution centered at zero. Since the model is linear, the SHAP values primarily capture monotone linear effects.

High-dimensional scenario Keep the setting and covariates are the same as low-dimensional case, but increase the dimension to 500 and make it as sparse linear regression: $Y_i = \beta_0 + X_i^\top \beta + \varepsilon_i$, with $\beta_0 = 1.5$, $\beta = (0.5, 2.0, -1.0, 1.5, -1.0, 0, \dots, 0)^\top$.

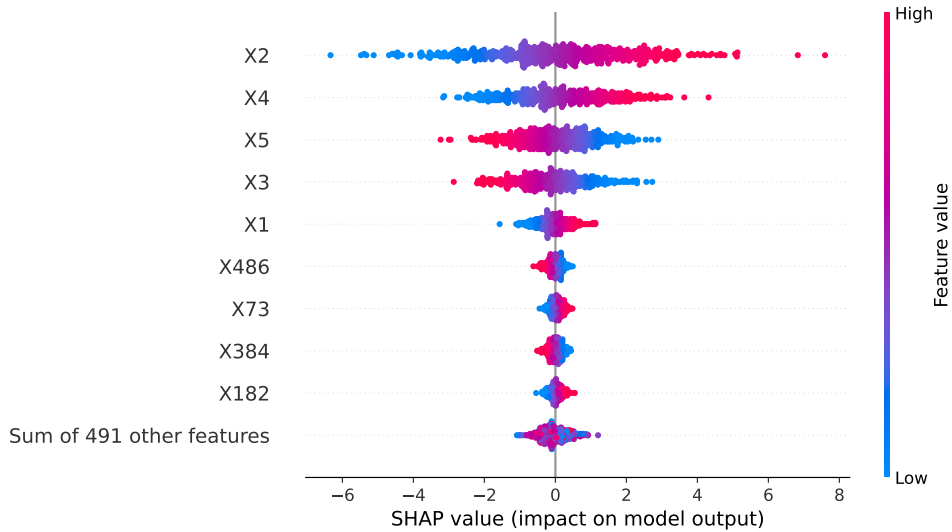


Figure 4: Beeswarm Plot ($p=500$, $n=500$)

- In the high-dimensional setting, only a small subset of variables has substantial predictive contribution. The beeswarm plot correctly identifies X_2 , X_4 , X_5 , X_3 , and X_1 as the dominant features.

- The ranking of feature importance is consistent with the sparse signal structure:

X_2 has the largest effect, X_4 has the second largest effect,

while X_3 , X_5 , and X_1 have smaller but still visible effects.

- For X_2 and X_4 , high feature values correspond to positive SHAP values, indicating that larger values of these variables increase the predicted response. This is consistent with positive regression coefficients.
- For X_3 and X_5 , high feature values correspond to negative SHAP values, while low feature values correspond to positive SHAP values. This indicates negative regression effects.
- The variables X_{486} , X_{73} , X_{384} , and X_{182} have SHAP values tightly concentrated around zero. Their small spread suggests that they contribute little to the model output and are likely noise or weakly associated variables.
- The row labeled “Sum of 491 other features” is also concentrated near zero, showing that the large collection of remaining variables has negligible aggregate impact. This supports the sparsity assumption in the high-dimensional design.

3.2 Nonlinear-based

Under the nonlinear simulation setting, the data are generated from a nonlinear outcome model. We then fit both a linear regression model and a nonlinear regression model to the same data. The goal is to compare the resulting SHAP values across the two fitted models and illustrate that SHAP provides a model-based measure of feature importance. In other words, the SHAP explanations reflect the behavior of the fitted prediction model rather than the true data-generating mechanism directly.

Data Generating Procedure:

- **Sample size and replications** $n = 500$, $p = 5$, $n_{\text{reps}} = 100$
- **Covariate generation** $X_i \sim N_p(\mu, \Sigma)$, $i = 1, \dots, n$, where

$$\mu = (0, 0, 0, 0, 0)^\top, \quad \Sigma_{jk} = \rho^{|j-k|}, \quad j, k = 1, \dots, p.$$

- **Fixed design**

- The design matrix $X \in \mathbb{R}^{500 \times 5}$ is generated once.
- The same X is used across all replications.

- **Outcome model**

$$Y_i = \beta_0 + f_1(X_{i1}) + f_2(X_{i2}) + f_3(X_{i3}) + \varepsilon_i,$$

with intercept $\beta_0 = 1.5$, and component functions

$$f_1(x) = 2\log(|x|), \quad f_2(x) = 1.5\sqrt{x}, \quad f_3(x) = -2\log(|x|).$$

The remaining covariates X_4 and X_5 are inactive.

- **Noise distribution**

$$\varepsilon_i \sim N(0, \sigma_\varepsilon^2), \quad \sigma_\varepsilon = 1.$$

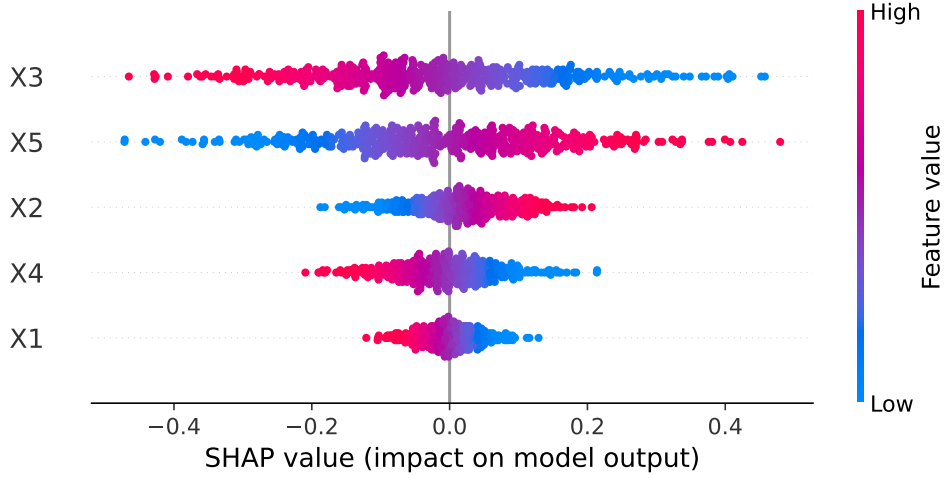


Figure 5: Beeswarm Plot (Under Linear Model)

- The true data-generating mechanism is nonlinear and depends only on X_1 , X_2 , and X_3 , while X_4 and X_5 are inactive. However, the fitted model is linear. Therefore, the SHAP values explain the fitted linear approximation rather than the true nonlinear mechanism.
- The beeswarm plot shows that X_3 has the largest SHAP dispersion, suggesting that the linear model captures part of the nonlinear signal from $f_3(X_3)$.
- The active variable X_2 also shows a visible contribution, but its SHAP spread is relatively small, indicating that the linear model only partially captures the nonlinear effect $f_2(X_2) = 1.5\sqrt{X_2}$.
- The active variable X_1 has a very small SHAP spread, even though it is truly relevant through $f_1(X_1) = 2\log(|X_1|)$. This suggests that the linear model fails to represent this nonlinear effect well.
- The inactive variables X_4 and X_5 receive non-negligible SHAP values, especially X_5 . This reflects model misspecification and possible correlation-induced attribution, rather than true causal or structural importance.
- Overall, this example illustrates that SHAP is model-dependent: when the fitted model is misspecified, SHAP explains how the misspecified model uses the covariates, not necessarily which variables are truly active in the data-generating process.

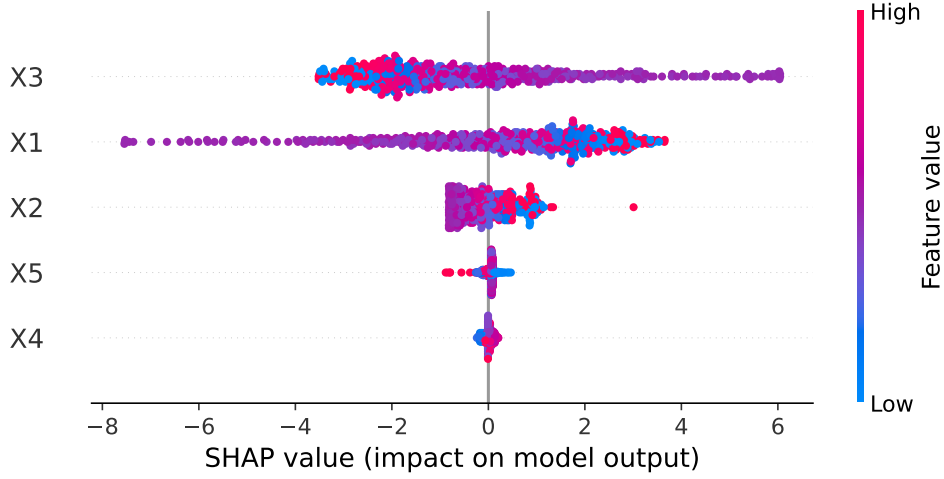


Figure 6: Beeswarm Plot (Under fitted nonlinear model)

- The true data-generating model is nonlinear and depends on X_1 , X_2 , and X_3 , while X_4 and X_5 are inactive. Under the nonlinear fitted model, the beeswarm plot correctly assigns the largest SHAP dispersion to the active variables X_3 , X_1 , and X_2 .
- The inactive variables X_4 and X_5 have SHAP values concentrated near zero, indicating that the nonlinear model largely recognizes their negligible contribution.
- The SHAP patterns for X_1 and X_3 are more irregular than in the linear case. This is expected because their true effects are logarithmic:

$$f_1(x) = 2 \log(|x|), \quad f_3(x) = -2 \log(|x|).$$

Since these functions depend on $|x|$, both large positive and large negative feature values can produce large effects, leading to mixed colors on both sides of the SHAP axis.

- For X_2 , the SHAP values are mostly positive but have a smaller spread, suggesting that the nonlinear model captures its effect

$$f_2(x) = 1.5\sqrt{x},$$

but the effect is weaker or more restricted than those of X_1 and X_3 .

- The apparent messiness of the points does not necessarily indicate a poor explanation. Instead, it reflects nonlinear, non-monotone feature effects and possible interactions learned by the boosted model.

Overall, comparing with the previous plot, the plot supports the idea that SHAP explanations depend strongly on the fitted model: when the model is flexible enough to capture the nonlinear structure, SHAP better reflects the true active features, but the visual pattern becomes less simple than in a linear model.

4 Discussion

Statistical inference for the SHAP curve. SHAP uses the concept of Shapley values from cooperative game theory to fairly allocate credit to the features in a vector $X \in R^p$ based on their contribution to an outcome Y . While the explanation offered by SHAP are local by nature, people often need global measures of feature importance in order to improve model explainability and perform feature selection. Whitehouse et al. (2026) [6] studies how to conduct statistical inference for SHAP-based variable importance. The key contribution of this paper is to develop inferential tools for the population-level SHAP curve and its global summaries. Let $\phi_a(x)$ denote the SHAP curve for feature a , which measures the contribution of feature a to the population regression function $\mu_0(x) = \mathbb{E}(Y \mid X = x)$. Since $\phi_a(x)$ is a local explanation depending on the covariate value x , the paper focuses on global summaries of the SHAP curve, especially

$$\theta_p = \mathbb{E} [|\phi_a(X)|^p],$$

where $p = 1$ corresponds to mean absolute SHAP and $p = 2$ corresponds to mean squared SHAP. These quantities are widely used as global feature-importance measures, but standard SHAP analysis usually reports them without uncertainty quantification. The main challenge is that ϕ_a is unknown and must be estimated from data, since it depends on the unknown regression function μ_0 . A naive plug-in estimator based on $\hat{\phi}_a$ may suffer from first-order bias and would require unrealistically fast nuisance-estimation rates to obtain valid asymptotic normality. To address this issue, the paper constructs de-biased U-statistic estimators based on Neyman-orthogonal scores. These scores include correction terms that remove the first-order effect of estimating nuisance functions such as μ_0 and ϕ_a . For $p \geq 2$, the resulting estimator is shown to be asymptotically linear and asymptotically normal, which enables valid confidence intervals for θ_p . For $1 \leq p < 2$, including the practically important case of mean absolute SHAP, the target functional is non-smooth because the map $u \mapsto |u|^p$ is not twice differentiable at zero. The paper handles this irregularity by introducing a smooth approximation to $|u|^p$ and carefully controlling the smoothing bias. In addition, the paper proposes a Neyman-orthogonal loss for learning the entire SHAP curve $\phi_a(x)$ through empirical risk minimization. Overall, the paper provides a rigorous framework for attaching confidence intervals to SHAP-based global importance measures and emphasizes that SHAP curves can be treated as statistical objects requiring both estimation and uncertainty quantification.

Bootstrap estimation of the SHAP curve. Another potential idea is to use the bootstrap to quantify the variability of the estimated SHAP curve. For a given feature j and observation x_i , the local SHAP value $\phi_j(x_i)$ measures the contribution of feature j to the prediction at x_i . Since $\phi_j(x_i)$ is estimated from a fitted model and therefore depends on the observed training sample, it naturally carries sampling uncertainty. To capture this uncertainty, we could repeatedly generate bootstrap samples from the original data, refit the prediction model on each bootstrap sample, and recompute the SHAP value for the same test point x_i . This produces a bootstrap distribution

$$\hat{\phi}_j^{(1)}(x_i), \hat{\phi}_j^{(2)}(x_i), \dots, \hat{\phi}_j^{(B)}(x_i),$$

where B is the number of bootstrap replications. The bootstrap mean can be used as a stabilized estimate of the SHAP curve at x_i ,

$$\bar{\phi}_j(x_i) = \frac{1}{B} \sum_{b=1}^B \hat{\phi}_j^{(b)}(x_i),$$

and the empirical quantiles of the bootstrap distribution can be used to construct pointwise confidence intervals. In this way, the SHAP curve is treated as an estimable statistical object rather than a fixed output of a single fitted model. This approach could provide a practical way

to visualize both the estimated feature contribution and its uncertainty across the covariate space.

Comparison between SHAP and penalized variable selection methods. It is also useful to distinguish SHAP-based feature importance from penalized variable selection methods such as LASSO, SCAD, and MCP. Penalized methods perform variable selection during model fitting by adding a regularization term to the empirical loss. For example, LASSO solves

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{n} \sum_{i=1}^n \ell(y_i, x_i^\top \beta) + \lambda \|\beta\|_1 \right\},$$

and selects variables whose estimated coefficients are nonzero. In contrast, SHAP is usually applied after a model has already been fitted. It explains how the fitted model uses each feature by decomposing the model prediction into additive feature contributions. Therefore, penalized methods answer the question “which variables should be selected in the model?”, while SHAP answers the question “how much does each feature contribute to the prediction made by a fitted model?”

A key difference is that LASSO, SCAD, and MCP are model-estimation and variable-selection procedures, whereas SHAP is an explanation and importance-assessment procedure. Penalized regression methods are closely tied to the assumed model class, such as linear or generalized linear models, and their selected variables depend on the penalty level λ , the penalty shape, and the correlation structure among covariates. SHAP, on the other hand, can be applied to a broad range of fitted models, including linear models, tree-based models, and neural networks. This makes SHAP more flexible for interpreting complex nonlinear prediction models. However, SHAP does not by itself produce a sparse model or remove variables from the fitted model; it only measures their contribution after fitting.

Another important distinction concerns interpretation. In a linear model, coefficient-based selection and SHAP importance may be closely related, because the contribution of feature j is largely driven by the product $x_j \hat{\beta}_j$. In this case, variables with large nonzero coefficients often have large SHAP values. However, in nonlinear models, SHAP can capture nonlinear effects and interactions that coefficient-based methods may miss. For example, a variable may have a small or unstable linear coefficient but still produce large SHAP values if its effect is nonlinear or depends on other features. Conversely, an inactive or weakly relevant variable may receive non-negligible SHAP attribution if the fitted model uses it indirectly through correlation with other predictors. Thus, SHAP importance should be interpreted as model-based predictive contribution rather than as direct evidence of true structural variable relevance.

From a statistical perspective, penalized methods are often studied through selection consistency, estimation error bounds, and oracle properties. Their goal is to recover the true active set under assumptions such as sparsity, restricted eigenvalue conditions, or irrerepresentable conditions. SHAP analysis has a different inferential target: it aims to estimate local or global feature contribution functions, such as the SHAP curve $\phi_j(x)$ or global summaries like $\mathbb{E}|\phi_j(X)|$. Therefore, SHAP and penalized methods are complementary. Penalized methods can be used to construct a sparse and stable predictive model, while SHAP can be used afterward to explain the selected model and visualize how each selected feature affects predictions.

References

- [1] David Alvarez-Melis and Tommi S. Jaakkola. “On the Robustness of Interpretability Methods”. In: *Proceedings of the ICML Workshop on Human Interpretability in Machine Learning*. 2018.

- [2] Scott M. Lundberg and Su-In Lee. “A Unified Approach to Interpreting Model Predictions”. In: *Advances in Neural Information Processing Systems*. 2017.
- [3] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. ““Why Should I Trust You?”: Explaining the Predictions of Any Classifier”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1135–1144.
- [4] Lloyd S. Shapley. “A Value for n-Person Games”. In: *Contributions to the Theory of Games II*. Ed. by Harold W. Kuhn and Albert W. Tucker. Vol. 28. Annals of Mathematics Studies. Princeton, NJ: Princeton University Press, 1953, pp. 307–317.
- [5] Erik Štrumbelj and Igor Kononenko. “Explaining Prediction Models and Individual Predictions with Feature Contributions”. In: *Knowledge and Information Systems* 41.3 (2014), pp. 647–665.
- [6] Justin Whitehouse, Ayush Sawarni, and Vasilis Syrgkanis. “Statistical Inference and Learning for Shapley Additive Explanations (SHAP)”. In: *arXiv preprint arXiv:2602.10532* (2026).